

COMPANION
to

ChatGPT Teaches TikZ

*Chapter 23: Loops, Lists, and Computed
Drawings (A)*

Contents

23.1	What This Companion Adds	2
23.2	A Computation Vocabulary	3
23.3	From Copies to a Rule	3
23.4	Lists as Small Data Tables	4
23.5	Computing Circular Positions	4
23.6	Calculating During a Loop	5
23.7	Integer Indices	5
23.8	Nested Loops	6
23.9	Making the Loop Range Depend on Data	6
23.10	Worked Project: A Regular Polygon Graph	7
23.11	Testing Computed Drawings	9
23.12	Debugging Workshop	9
23.13	Exercises	10
23.14	Solutions	10
23.15	ChatGPT Workshop	12
23.16	Final Checklist	12

23.1 What This Companion Adds

Chapter 23 used loops and PGF mathematics to express repeated drawing rules. This Companion develops a data-first workflow: store attributes together, use integer calculations for names and indices, and test the rule on a small case before generating a larger diagram.

Design

Use a loop when the source becomes a clearer statement of the mathematical pattern. Shorter code alone is not sufficient reason.

23.2 A Computation Vocabulary

Form or command	Purpose
<code>\foreach</code>	Repeats commands for values in a list.
<code>{0,...,5}</code>	Generates a regular numerical sequence.
<code>\x/\label</code>	Splits paired data into variables.
<code>evaluate=</code>	Computes a loop value during iteration.
<code>\pgfmathsetmacro</code>	Stores a numerical calculation.
<code>\pgfmathtruncatemacro</code>	Stores an integer calculation.
<code>mod(a,b)</code>	Computes a remainder for cyclic indices.
<code>cos, sin</code>	Compute circular coordinates in degrees.

23.3 From Copies to a Rule

```
\foreach \x in {0,...,4}
  \fill (\x,0) circle (2pt);
```

The list is data; the drawing command is the rule applied to each item.

See



Five unrelated points should not be forced into this form. Five equally spaced points naturally belong to one loop.

23.4 Lists as Small Data Tables

```
\foreach \id/\x/\y/\label in {  
  A/0/0/$A$,  
  B/2/1/$B$,  
  C/4/0/$C$,  
  D/2/-1/$D$  
}  
  \node[draw,circle] (\id) at (\x,\y) {\label};
```

The internal name, coordinates, and printed label travel together. Keep internal names simple even when visible labels contain mathematics.

23.5 Computing Circular Positions

```
\def\radius{2}  
\foreach \a in {0,60,...,300}  
{  
  \pgfmathsetmacro\x{\radius*cos(\a)}  
  \pgfmathsetmacro\y{\radius*sin(\a)}  
  \fill (\x,\y) circle (2pt);  
}
```

The coordinates are consequences of the rule rather than manually entered approximations.

23.6 Calculating During a Loop

The `evaluate` option keeps a short calculation close to the loop variable.

```
\def\n{6}
\foreach \k [
  evaluate=\k as \angle using {90-360*\k/\n}
] in {0,...,5}
  \coordinate (v\k) at (\angle:2);
```

Use `\pgfmathsetmacro` when several commands need the result or when the calculation is easier to read on its own line.

23.7 Integer Indices

Node names such as `v3` require integer values. Use a truncating macro for the next cyclic index.

```
\pgfmathtruncatemacro{\next}{mod(\k+1,\n)}
\draw (v\k)--(v\next);
```

Without truncation, a numerical result may contain a decimal form unsuitable for the intended node name.

Common Mistake

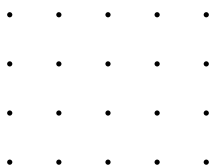
Distinguish visual numbers from structural indices. A coordinate may use a decimal; a node-name suffix or list index often must be an integer.

23.8 Nested Loops

```
\foreach \x in {0,...,4}
  \foreach \y in {0,...,3}
    \fill (\x,\y) circle (1.5pt);
```

Two loops are natural because each grid point is determined by one column and one row.

See



A 5x4 grid of dots, representing a grid of points. The dots are arranged in 5 rows and 4 columns, forming a rectangular pattern.

Nested loops also express complete bipartite adjacency: choose one vertex from the left family and one from the right family.

23.9 Making the Loop Range Depend on Data

Avoid defining `n=8` while leaving an unrelated literal upper bound 5.

```
\def\n{8}
\pgfmathtruncatemacro{\last}{\n-1}
\foreach \k in {0,...,\last}
  ...
```

Now changing `n` updates the range as well as the angle calculation.

23.10 Worked Project: A Regular Polygon Graph

23.10.1 Stage 1: Generate the Vertices

```
\def\n{6}
\def\radius{2.2}
\pgfmathtruncatemacro{\last}{\n-1}
\foreach \k in {0,...,\last}
{
  \pgfmathsetmacro{\angle}{90-360*\k/\n}
  \node[vertex] (v\k) at (\angle:\radius) {$v_{\k}$};
}
```

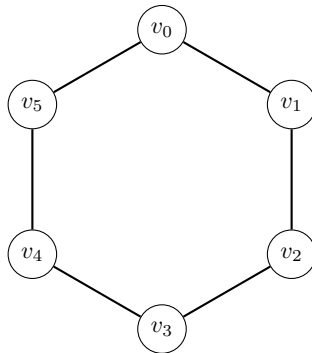
23.10.2 Stage 2: Generate the Cycle Edges

```
\foreach \k in {0,...,\last}
{
  \pgfmathtruncatemacro{\next}{mod(\k+1,\n)}
  \draw[edge] (v\k)--(v\next);
}
```

23.10.3 The Complete Picture

```
\begin{tikzpicture}[
  vertex/.style={draw,circle,fill=white,minimum size=7mm},
  edge/.style={thick}
]
  \def\n{6}
  \def\radius{2.2}
  \pgfmathtruncatemacro{\last}{\n-1}
  \foreach \k in {0,...,\last}
  {
    \pgfmathsetmacro{\angle}{90-360*\k/\n}
    \node[vertex] (v\k) at (\angle:\radius) {$v_{\k}$};
  }
  \foreach \k in {0,...,\last}
  {
    \pgfmathtruncatemacro{\next}{mod(\k+1,\n)}
    \draw[edge] (v\k)--(v\next);
  }
\end{tikzpicture}
```

See



23.11 Testing Computed Drawings

Test $n = 3$, the ordinary case, and one larger value. Check the first and last indices, node-name uniqueness, the closing edge, and whether any calculation should be integral. Small boundary cases expose most loop errors quickly.

23.12 Debugging Workshop

23.12.1 The Last Vertex Does Not Connect Back

Use `mod(k+1,n)` for a cyclic next index.

23.12.2 A Node Name Contains a Decimal

Replace a general math macro with `\pgfmathtruncatemacro` for the structural index.

23.12.3 Changing n Leaves Six Vertices

Derive the loop's last index from `n`; remove copied numerical bounds.

23.12.4 The Loop Is Harder to Read than the Figure

Use an explicit data list or several clear drawing commands when the repetition does not follow one simple rule.

23.13 Exercises

1. Generate five equally spaced marks.
2. Place named vertices from a multi-field data list.
3. Compute eight points on a circle.
4. Generate a rectangular array with nested loops.
5. Draw a complete bipartite graph with two loops.
6. Generate an n -cycle from one value of n .
7. Add every second chord to a regular polygon graph.
8. Test a computed drawing at its smallest valid input.

23.14 Solutions

23.14.1 Exercises 1–4

```
\foreach \x in {0,...,4} \fill (\x,0) circle (2pt);
\foreach \k in {0,...,7}
{
  \pgfmathsetmacro{\a}{45*\k}
  \fill (\a:2) circle (2pt);
}
```

Exercise 2 uses one field each for internal name, position, and label. Exercise 4 nests independent row and column loops.

23.14.2 Exercises 5–7

For Exercise 5, the outer loop selects a left vertex and the inner loop selects a right vertex. Exercise 6 uses the worked project’s modular next index.

```
\pgfmathtruncatemacro{\next}{mod(\k+2,\n)}  
\draw (v\k)--(v\next);
```

Exercise 8 tests the first and last index and verifies that every referenced node was created.

23.15 ChatGPT Workshop

Ask ChatGPT to preserve the mathematical rule while reviewing the program.

1. Replace copied commands with a loop only where a clear rule exists.
2. Convert these manual coordinates into readable PGF calculations.
3. Check every loop bound and cyclic index for off-by-one errors.
4. Identify calculations that must use an integer macro.
5. Test this computed drawing on small and boundary input values.

23.16 Final Checklist

Before continuing to Chapter 24, check that you can

1. distinguish genuine repetition from unrelated copies;
2. organize several attributes in one data list;
3. calculate coordinates from a mathematical rule;
4. choose between general and integer math macros;
5. use nested loops for two independent choices;
6. derive bounds from the controlling data; and
7. test computed diagrams at boundary cases.